

S-38.3159 Assignment 1: Protocol specification

Hämäläinen, Harri Islam, Hasan
`harri.hamalainen@aalto.fi` `hasan.islam@aalto.fi`

Kurnikov, Arseny
`arseny.kurnikov@aalto.fi`

November 20, 2014

Contents

1	Revision log and open issues	2
1.1	Open issues	2
1.2	Revision log	2
2	Introduction	3
3	Protocol mechanisms	4
4	Protocol encoding	6
4.1	Message format	6
5	Protocol messages	8
5.1	Listing available sensors	8
5.2	Subscribing	8
5.3	Receiving updates	9
5.4	Unsubscribing	10
5.5	Keep-alive messaging	10
6	Congestion control	11
6.0.1	Skipping messages	11
6.0.2	Throttling down the sending rate	12
6.0.3	After thoughts about about our congestion control design	12

Chapter 1

Revision log and open issues

1.1 Open issues

- Protocol version field
- How to rate limit the sending of updates
 - How much to increase / decrease the effect of actions taken to deal with the congestion

1.2 Revision log

12. February 2013	Initial release
26. March 2013	Added a chapter about congestion control
31. June 2013	Updated the spec to reflect what we were after to with congestion
2. May 2013	Updated the messages to also include NACK messages

Chapter 2

Introduction

This specification describes a publish-subscribe protocol for Internet-of-Things (IoT) client-server exchange. In the typical scenario there is a set of sensors that send periodical updates to a server. The server should collect sensor data while accepting subscriptions from clients that are interested in particular subset of sensor updates. Once a client is subscribed to some sensors it should receive updates the same way as the server receives them from the sensors. When the client is no longer willing to receive updates from the server it should unsubscribe from the updates.

The protocol specification described in this document has a few design goals which one should be familiar of to understand the design choices made. First of all to our understanding the IoT style communication is going to be used with multiple sensors and multiple clients. The sensors have different characteristics; they send updates at different rates, the size of the updates vary as does their content types. We are not to limit what kind of data the sensors can produce. Another key design choice made is not to provide fully reliable transmission. IoT communication patterns are usually making of use the most recent data and offering full reliability service is out of the scope of this specification.

Chapter 3

Protocol mechanisms

There are currently four types of sensors supported by the protocol: a device, a temperature sensor, a GPS sensor and a camera. Their functionality is as follows.

A device sensor sends aperiodical updates about its status, that can be on or off. A temperature sensor sends periodical updates (every second) about the temperature. A GPS sensor is also aperiodical. Finally a camera sensor might send no data at some time periods with bursts of binary data when the sensor detects some motion.

As can be seen there are two fundamentally different classes of sensors: the ones with periodical and often updates (like a camera and a temperature sensor), the others are aperiodical and comparatively rarely update the data. The specification is currently limited to the four aforementioned sensor types but it makes the main difference between the two sensors classes. Mainly the difference lies in reliability mechanisms.

The protocol discussed here is supposed to serve many different kind of sensors. Basically the sensors are serving the state information of the sensor or some device the sensor is monitoring. Among different users of this protocol it is likely that we encounter some requirements for reliability of the protocol. In these cases the protocol should be able to deliver the updates to clients even while some messages might be lost. Such a situation occurs for example with simple status updates: if a sensor is in "OFF" state and next publishes its status to be "ON" but the message is lost the client sees no change in the state of the sensor. The reliability with this kind of messages is required as we see this kind of devices like state machines and we also point out that in real deployment the client takes some action based on the received status messages. Furthermore unreliable transfer of status messages and other messages with similar requirements leads to situation where one user might think the device is in "OFF" state while some others see it in "ON" state.

In some cases it is likely that a loss of some updates can be tolerated. Such is the case for example with streaming data i.e. updates that are soon outdated. In these cases it is likely that the reliability guarantee will only lead to wasted resources.

To manage these requirements the client can specify in a **subscribe** message a specific flag requiring the updates to be delivered in reliable manner. When this reliability parameter is used the server requires that client acknowledges

updates. If no acknowledgment is received within a given timeout the server will retransmit the update message to the client. However the protocol does not guarantee that all messages will be delivered to clients when this optional reliability is used. If a new update is received by the server from a sensor and there are still some active retransmissions to clients from this same sensor the newer update will replace the old message. We refer this behaviour as *weak-reliability*. We argue that for IoT protocol like this the weak-reliability is enough as many kind of applications need only the most recent information. Applications requiring more strict reliability guarantees should fall back using plain old TCP connections.

In order to avoid reordering of messages all subscriptions have a logical time value which is incremented with each message. A client receiving an update from a sensor with smaller time value than the previous update from this same sensor must discard the message as old. When the timer value of the last received message is high enough the receiver should accept also timer values with very small timer values. This is because the timer value is basically an unsigned integer which will overflow and reset to zero when the value grows big enough.

The server keeps a list of subscribed clients per sensor and the list used when an update must be delivered to subscribed clients. However because the publish-subscribe protocol uses UDP on transport layer the server has no information whether or not the updates are reaching the client when the connection is specified as unreliable. In order to avoid server to still handle "dead clients" in its receiver lists a client must eventually send **keep-alive** messages to server. The keep-alive timeout is a protocol parameter used by both servers and clients. The server should use a a client keep-alive timeout multiplied by some constant to avoid unnecessary client removals. If a client is removed from the server it no longer receives updates from the server and it must resubscribe with the normal subscribe procedure.

The recommended value for the client keep-alive timeout is 5 seconds. And the recommended value for the server constant is 5. Thus if no keep-alive is received within 25 seconds, the client is pruned from the client list.

Chapter 4

Protocol encoding

The protocol encoding used in this specification for the communication between the server and the clients is a subset of BSON ¹. BSON is a binary format in which zero or more key/value pairs are stored as a single entity. BSON is used for storing JSON-like data in binary form.

The communication between sensors and clients through the server is composed of two parts: between sensors that publish data and the server, and between the server and the clients that subscribe to the data. It is important to have a suitable encoding for data transmission. JSON is used on communications between the server and the sensors.

4.1 Message format

Here are the message examples in JSON format that become BSON after binary encoding. Exact format specification is in the following chapter.

```
{'type': 2, 'sids': [{'id': 'gps_1', 'reliable': true},  
{'id': 'camera_1', 'reliable': false}]}
```

```
{'type': 3, 'sids': [{'id': 'gps_1', 'status': 200},  
{'id': 'camera_1', 'status': 404}]}
```

The key `type` indicates the type of the message. There are the following message types.

0. Sensors list request
1. Sensors list reply
2. Subscribe
3. Subscribe acknowledgment
4. Update
5. Update acknowledgment

¹<http://bsonspec.org>

6. Unsubscribe

7. Keep-alive

Chapter 5

Protocol messages

In this chapter we describe different protocol messages in detail. For readability the protocol messages are presented before their presentation is encoded into BSON.

5.1 Listing available sensors

With `list` message type a client can retrieve a list of available sensors publishing updates into the server. The request has a following presentation:

```
{'type': integer}
```

The value of `type` key for list message must contain an integer value 0.

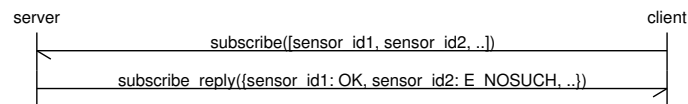
The `list response` message contains a list of available sensor IDs and has a following presentation:

```
{'type': integer, 'sids': [string, ...]}
```

The value of `type` key for list response message must contain an integer value 1. The value of `sids` key contains a list of strings interpreted as sensor IDs.

5.2 Subscribing

In order to receive updates from a specific sensor the client must subscribe to that sensor with a `subscribe` message. The message takes a list of elements describing the sensor id and whether or not the updates are to be delivered in a reliable manner.



The subscribe message sent by the client has the following format:

```
{'type': integer, 'sids': [{ 'id': string, 'reliable': integer }, ...]}
```

The value of **type** key for subscribe message must contain an integer value 2. The value of **sids** key contains a list of sensor objects. Each of the sensor objects have a key **id** with a string of the sensor id as a value and optional **reliable** key with a boolean value.

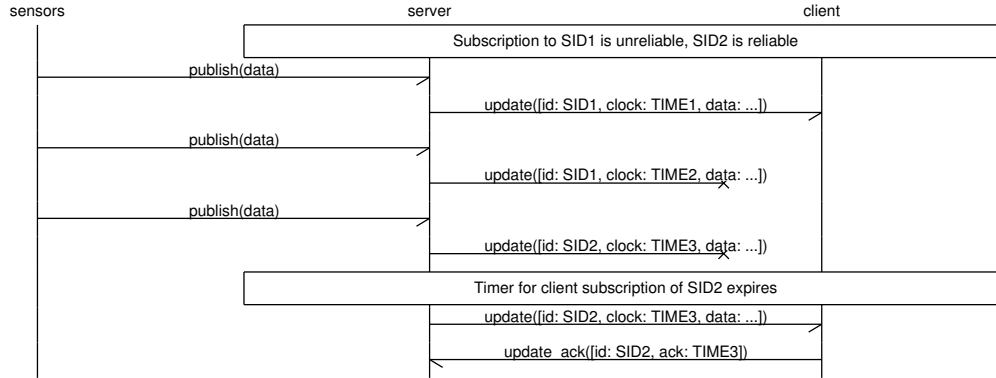
The subscribe acknowledgement from the server has the following format:

```
{'type': integer, 'sids': [{ 'id': string, 'status': integer}, ...]}
```

The value of **type** key for subscribe acknowledgement must contain an integer value 3. The value of **sids** key contains a list of sensor objects. Each of the sensor objects have a key **id** with a string of the sensor id as a value and **status** key with an integer value as a return value. If the subscription was successful the return value must be 0. Other values indicate different error reasons. When the sensor does not exist the return value is 1. If the client is already subscribed, the return value is 2.

5.3 Receiving updates

The updates from sensors are delivered to clients with **update** messages. Depending on the subscribe parameters the update delivery can be either reliable or unreliable.



The message has the following format before it is encoded into a binary form:

```
{'type': integer, 'sid': string, 'sensor_type': integer, 'time': integer, 'reliable': boolean, 'data': sensor_data}
```

The value of **type** key for update message must contain an integer value 4. The value of **sid** key is a string presentation of the sensor id. the **sensor_type** key is used to hold the information about the type of the sensor. Possible values are 0 for status sensors, 1 for temperature sensors, 2 for GPS sensors, and 3 for camera sensors. The value of **time** key holds an integer value (protocol clock) used with reliable transport. The value of **reliable** key indicates whether or not the client must acknowledge this update. The value of **data** contains the actual data from sensor. This can be boolean (status sensors), float (temperature sensors), string (GPS), or binary (camera) data depending on the sensor type of the message.

If the transmission is reliable and it is stated in the update message then the client must send the update acknowledgment.

The update acknowledgment message has the following format:

```
{'type': integer, 'sid': string, 'sensor_type': integer, 'time': integer}
```

The value of **type** key for the update acknowledgment message must contain an integer value 5. The value of keys **sid**, **sensor_type** and **time** must be the same as in the update message.

The client may also support negative acknowledgements which are sent when the client detects a packet loss from cap in the message sequence numbers. This message is sent to server to inform it that a loss has occurred.

The negative update acknowledgment message has the following format:

```
{'type': integer, 'sid': string, 'seq_no': integer}
```

5.4 Unsubscribing

When a client wants to end the receiving of updates from a specific sensor it sends a **unsubscribe** message to the server. Upon receiving the **unsubscribe** message from a client the server removes the client from the recipient list of the specific sensor IDs given in the unsubscribe message.

The unsubscribe message sent by the client has the following format:

```
{'type': integer, 'sids': [{'id': string}, ...]}
```

The value of **type** key for the unsubscribe message must contain an integer value 6. The value of **sids** key contains a list of sensor objects. Each of the sensor objects have a key **id** with a string of the sensor id as a value.

5.5 Keep-alive messaging

The clients must send **keep-alive** messages with constant pace to server. From the absence of these keep-alive messages the server can infer the dead clients and can safely remove these from its receiver lists.

The keep-alive message sent by the client has the following format:

```
{'type': integer}
```

The value of **type** key for the keep-alive message must contain an integer value 7.

Chapter 6

Congestion control

The congestion control is used limit the transmission rate when the network is diagnosed to suffer from congestion. Several factors can be used to detected the congestion. Traditional TCP variants most often use packet losses as an indicator of congestion, some others measure the latencies and deduce the increased queueing delay as first signs of congestion. We can also intelligent middleboxes to implement ECN functionality, which adds additional signalling to inform end hosts about the congestion. We chose to use packet losses as it is the one which works reasonable well without large scale testing and analysis (which is most likely required to fine tune the delay based congestion control mechanishms) or without any additional mechanisms from the network.

The setting we have in this assignment assumes that we cannot control the sensor nodes so nothing can done to improve the behaviour under congestion on that link. So we study only the congestion on the link between the server and the clients. Basically the server is the network intensive part of the protocol implementation as it creates the most of the protocol messages under normal conditions.

The protocol has three communication patterns the server can use to measure and deduce that a loss has occured. The first one is a missing update acknowledgement when reliable subscription is used. The another one is to rely on negative acknowledgements sent by the client who can deduce a loss from missing sequence numbers and send the NACK message even if the communication flow is not set up to be reliable.

When a loss is detected the server must either *skip messages* or *throttle down* its sending rate. The first one is a more simple solution which causes the server simply to skip some updates from being delivered. The latter option will cause the server to send the updates at slower pace. Because we use weak reliability guarantees this also means that if the sensors publish updates at a pace greater than the server can send the updates forward, some of these updates will be discarded.

6.0.1 Skipping messages

Because flows without reliability guarantees are not communicating back to server server has no means of reacting quickly to congestion situations by default. One simple idea is to add negative acknowledgements into protocol design. This

is a mechanisms which is generally considered quite inferior as more messages are added into already congested network (e.g. see why ICMP source quench is deprecated).

When a client detects a gap in the sequence number flow the client will generate a special NACK packet and sends it to server. Server receiving the packet will deduce that a link is congested and will simply skip the next message send to client. When a message is skipped the server will reset the lost message state and the next message after the skipped message will be delivered normally.

Message skipping is happening on the link between the server and the client shared by multiple subscriptions.

6.0.2 Throttling down the sending rate

When a server detects congestion it can also implement more sophisticated methods to handle the situation like to throttle down its sending rate. This means that server will schedule the transmission of update later under the congestion. With our reliability guarantees this also means that a scheduled message might be replaced by a newer message which means that it's actually never transmitted.

6.0.3 After thoughts about about our congestion control design

Adding congestion control on top if this kind of application turned out to be quite taunting task. First of all we have a possibility of two kind of clients first of which are basically only receiving updates and never acknowledging these back to server which however require a method of informing server about the congestion. Another problem is the weak-reliability guarantee which adds its own complexity due to fact that not all messages are always even supposed to be received (e.g. in case of update bursts when a new message quickly replaces the old one).